tier2_cuttlefish_ab.sh — user guide

Field	Value
Revision	1
Last modified	2026-06-11T00:00:00Z
Status	active — host-gated (SKIPs cleanly off a Linux+KVM host)
Authority	Helix OTA control-plane / device-integration team
Related	`docs/design/CUTTLEFISH_TIER2.md` (the bring-up plan + sources), `docs/design/EMULATED_DEVICE_TESTING.md` (Tier-1/2/3 overview), `tests/emulator/ab_virt/` (the planned A/B-virt PWU-AB-3 this PWU-CF-2 case mirrors)

Overview

`tests/emulator/tier2_cuttlefish_ab.sh` exercises the **real Android `update_engine` A/B + AVB/dm-verity + auto-rollback** flow on a Cuttlefish (`cvd`) virtual Android device — the hardware-free fidelity tier the dev-host QEMU A/B-virt tier (`tests/emulator/ab_virt/`) cannot reach, and the closest proxy for the Orange Pi 5 Max / RK3588 OTA apply.

It runs in two phases on a Linux + nested-KVM host:

1. **Slot-flip phase** — `launch_cvd` → read baseline active slot → apply an OTA payload to the inactive slot via `update_engine` → reboot → assert the active slot **FLIPPED** + dm-verity is present on the new slot.
2. **Auto-rollback phase (PWU-CF-2, headline)** — corrupt the now-INACTIVE previous slot (mark it unbootable / fail dm-verity), reboot, and assert the device **AUTO-ROLLS-BACK** to the known-good active slot (active slot does not revert to the corrupted one; boot succeeds on the good slot; the rollback trace is captured). This mirrors the planned `ab_virt` PWU-AB-3 corrupt-slot rollback case.

Honest status (§11.4.6 / §11.4.123)

The **entire Linux flow — including the PWU-CF-2 auto-rollback section — is `UNVERIFIED-pending-Linux-host`.** It is implemented from the latest official AOSP docs (see `docs/design/CUTTLEFISH_TIER2.md` “Sources verified”), but has

not been executed on a real Linux+KVM host. The Helix dev host is macOS on Apple Silicon (`applehv` hypervisor, no `/dev/kvm`), so the script **SKIPs cleanly there** (exit 3, `topology_unsupported`) — never a fake PASS.

The auto-rollback section does **not** claim Android auto-rollback works until the script is run on the real host. The two AOSP-`UNCONFIRMED:` steps it depends on — the exact OTA-apply driver, and the exact safe corrupt-the-inactive-slot mechanism (legacy A/B direct write vs Virtual A/B COW/`snapuserd` path; `bootctl` availability) — are **detected + attempted at runtime and FAIL honestly** if they do not reproduce (§11.4.6 no-guessing, §11.4.123 rock-solid-proof-or-research, §11.4.133 verified-before-destructive-write).

Prerequisites

- **Topology (mandatory):** Debian-based Linux + usable `/dev/kvm` (nested-virt on a cloud instance, an M4+/macOS-15 nested-virt Mac, or a Linux box). On any host without it, the script SKIPs (exit 3). Verify per `CUTTLEFISH_TIER2.md` §3: `grep -c -w "vmx\|svm" /proc/cpuinfo` (x86) or `find /dev -name kvm` (ARM64).
- **Cuttlefish host packages:** `cuttlefish-base` installed (run `--prepare` once, then reboot — see Usage). Group membership in `kvm,cvdnetwork,render`.
- **Tooling:** `git`, `apt`/`dpkg`, `python3`, `adb`/`fetch_cvd` (from the cvd host package), ~30 GB disk, network (AOSP build fetch).

Usage examples

```
# One-time prerequisite install (builds + installs the cuttlefish
        debs; needs sudo),
# then REBOOT to load kernel modules + udev rules:
tests/emulator/tier2_cuttlefish_ab.sh --prepare
sudo reboot

# Normal run (Linux+KVM host) — runs the full slot-flip + auto-
        rollback flow:
tests/emulator/tier2_cuttlefish_ab.sh

# On this macOS dev host — honest SKIP (the only thing verifiable
        here):
tests/emulator/tier2_cuttlefish_ab.sh   # -> [SKIP] ...
        topology_unsupported ; exit 3
```

Environment

Var	Default	Meaning
`HELIX_CF_DIR`	`./.cuttlefish`	cuttlefish workdir (debs, cvd images)
`HELIX_CF_TARGET`	auto by arch (`aosp_cf_arm64_only_phone` / `aosp_cf_x86_64_only_phone`)	the cuttlefish build target

Var	Default	Meaning
HELIX_CF_EVIDENCE	docs/qa/<run-id>/cuttlefish_ab/	captured-evidence dir (§11.4.69/ §11.4.83)

Inputs / Outputs / Side-effects

- **Inputs:** topology (Linux+KVM), cuttlefish host packages, an OTA payload (`ota.zip` + `update_device.py`) for the apply phase.
- **Outputs:** captured evidence under the evidence dir — `kvm.txt`, `getprop_before.txt`, `update_engine.txt`, `slot_after.txt`, `verity.txt`, `virtual_ab.txt`, `corrupt_bootctl.txt` / `corrupt_dd.txt`, `corrupt_setactive.txt`, `slot_after_rollback.txt`, `rollback_trace.txt`, `rollback_logcat.txt`, `transcript.txt` — plus a `PASS` / `FAIL` / `SKIP` verdict on stdout.
- **Exit codes:** `0` = PASS (real flow ran green on a Linux+KVM host), `1` = FAIL (a real defect, or an UNVERIFIED-pending-host step that did not reproduce — never a fake PASS), `3` = SKIP (topology absent / prerequisites missing).
- **Side-effects:** boots a `cvd` virtual device; the auto-rollback phase performs a **bounded** corruption of the **inactive** slot only (one 4K block, or a `bootctl set-slot-as-unbootable` on the inactive index) — never the active/good slot (§11.4.133). Self-cleaning: `stop_cvd` runs on every exit path (`trap ... EXIT INT TERM`, §11.4.14).

Edge cases

- **No `/dev/kvm` (e.g. macOS):** SKIP exit 3, `topology_unsupported`. Verified on the macOS dev host.
- **`cuttlefish-base` not installed:** SKIP exit 3 with a pointer to `--prepare`.
- **`fetch_cvd` / OTA package absent:** the apply phase FAILs honestly (`UNVERIFIED-pending-host`), it does **not** fake a PASS.
- **`bootctl` shell command absent:** the corrupt phase falls back to a bounded dm-verity-fail write on the inactive `system<slot>` by-name partition; if neither is available it FAILs honestly (no silent skip-to-PASS).
- **Device hangs after corruption:** treated as FAIL (a hang is **not** a rollback PASS); the `rollback_trace.txt` is captured for host-side forensics.
- **Auto-rollback does not occur (boots the corrupted slot):** FAIL, not PASS.

Internal behaviour

Topology gate (Linux + readable/writable `/dev/kvm`) → arch→target selection → prerequisite/`--prepare` check → fetch+launch `cvd` → baseline slot → real `update_engine` apply → reboot + slot-flip assertion → dm-verity assertion → **PWU-CF-2:** detect A/B variant (`ro.virtual_ab.enabled`) → corrupt the inactive slot (`bootctl set-slot-as-unbootable`, else bounded dm-verity-fail write) → set the corrupted slot active to force the bad-boot path → reboot → assert boot succeeds on the known-good slot + capture the rollback trace. All `adb`/`curl`/loop waits are bounded; every PASS cites an evidence path (§11.4.69 `ab_pass_with_evidence` shape).

Related scripts

- `tests/emulator/ab_virt/` — the dev-host QEMU A/B-virt tier whose PWU-AB-3 corrupt-slot rollback case this PWU-CF-2 mirrors.
- `docs/design/CUTTLEFISH_TIER2.md` — the bring-up plan, sourced FACTs, and the `UNCONFIRMED:` verify-before-run register (§8) this script honours at runtime.

Last verified

- macOS dev host SKIP path (exit 3, `topology_unsupported`): verified 2026-06-11.
- `sh -n` + `bash -n` parse-clean (§11.4.67): verified 2026-06-11.
- Linux+KVM full flow (slot-flip + auto-rollback): **UNVERIFIED-pending-Linux-host** — to be run when the operator's Linux+KVM host is attached.